

要把经典的SA（结构化分析）八步法真正“物理装配”到我们现有的LLM调用架构中，核心在于不能指望通过一个超长的Prompt让大模型一次性“悟”出八步法，而是要把八步法拆解为由代码控制的“多智能体串行流水线”，并用程序级的校验器锁死每一步的进出口。

这需要我们现有的需求分析阶段（Stage 1）进行重构。以下是具体的系统实现方案，分为架构重构、流水线编排、硬约束校验和提示词设计四个落地步骤。

第一步：架构重构——将单Agent拆解为SA子流水线

我们现有的 RequirementAnalysisAgent 是一个黑盒，接收文本输出JSON。我们需要将其拆解为一个编排器和8个专职子智能体。

在 TypeScript 中，我们可以这样定义这个子流水线：

```
// SA 流水线编排器
class SAOrchestrator {
  private llm: BaseLLM;
  private context: AgentContext;
  async run(rawRequirement: string): Promise<SADocument> {
    // Step 0: 边界与事件提取
    const scope = await this.callSAAgent('ScopeAgent', rawRequirement);
    await this.humanConfirm(scope); // 门控：客户必须确认核心事件不遗漏
    // Step 1&2: DFD 分层
    const dfd = await this.callSAAgent('DFDAgent', scope);
    await this.validate(new DFDValidator(dfd)); // 代码校验父子图平衡与数据守恒
    // Step 2.5: 业务流程图
    const bpm = await this.callSAAgent('BPMAgent', dfd);
    await this.validate(new BPMValidator(bpm, dfd)); // 校验流程节点与DFD过程映射
    // Step 3: 数据字典（最关键）
    const dict = await this.callSAAgent('DataDictAgent', {dfd, bpm, context:
this.context});
    await this.validate(new DictValidator(dict, dfd)); // 校验所有数据流都有字典定义
    // Step 4&5: PSPEC 与 判定表
    const logic = await this.callSAAgent('LogicAgent', {dfd, dict});
    await this.validate(new LogicValidator(logic, dict)); // AST 解析伪代码，校验字段引用
    // Step 6&7: ER 图与状态机
    const design = await this.callSAAgent('DesignAgent', {dict, bpm});

    // Step 8: UI 原型
    const ui = await this.callSAAgent('UIAgent', {bpm, dict, design});
    await this.validate(new UIValidator(ui, dict)); // 校验UI字段不超出字典范围
    return { scope, dfd, bpm, dict, logic, design, ui };
  }
}
```

第二步：流水线编排——六问清单与Event级路由

为了避免60个业务事件导致Token爆炸，我们必须在编排器中实现您参考文件提到的“三级分层”和“六问清单”机制。

当系统通过 Step 0 识别出 60 个事件后，SAOrchestrator 会对事件进行分类路由：

1. **Project级（仅执行一次）**：系统初始化时，引导 AI 生成全局 DFD 主图、核心数据字典骨架（如工单、用户表的80个公共字段）。
2. **Event级（针对每个具体事件）**：对这 60 个事件，并行（或串行）调用 EventAgent，执行“六问清单”。AI 此时不需要重画 DFD，只需回答“是否增量？增加几个节点？增加几个字段？”
3. **Process级（针对极复杂逻辑）**：当“六问”发现事件涉及复杂状态扭转（如物料倒冲），才触发完整的 Step 4 (PSPEC) 和 Step 5 (判定表) 深度推演。

第三步：硬约束校验器——这才是真正的“套上枷锁”

这是整个方案的核心。大模型的幻觉无法通过 Prompt 根除，只能通过代码拦截。我们需要编写一系列 validator。

1. DFD 守恒校验器

- **规则**：检查 0 层图的每个加工过程，输入数据流和输出数据流的字段合集必须匹配（无黑洞、无奇迹）。
- **实现**：遍历 DFD JSON 节点，比对 inputFlows 和 outputFlows 的字段定义。

2. 业务流程图与DFD映射校验器

- **规则**：业务流程图里的每一个活动节点，必须绑定一个 DFD 里的加工过程ID。
- **实现**：遍历 bpm.nodes，检查 node.bindProcessId 是否在 dfd.processes 中存在。如果 AI 画了一个 DFD 里没有的过程，直接报错打回。

3. 字典覆盖校验器

- **规则**：DFD 中出现的所有数据流箭头，必须在数据字典中有明确定义；ER 图的表字段，必须完全来源于数据字典。
- **实现**：提取 DFD 的 flowNames，去 DataDictionary 里查找，查不到则报错。

4. UI 字段越界校验器

- **规则**：生成的 UI 表单字段，必须 100% 来源于该业务流程绑定的数据流字典。
- **实现**：提取 UI JSON 里的 formFields，与 DataDictionary 对应的结构对比。如果 AI 幻觉生成了一个字典里没有的 scrapReason 字段，编译期直接中断。

第四步：提示词设计——将大模型降维为“受限填空机”

在 SA 流水线中，每个子 Agent 的 Prompt 不再要求“设计一个系统”，而是要求“根据提供的前置上下文，完成当前步骤的结构化填空”。

以最核心的 DataDictAgent（第三步）为例，Prompt 应这样设计：

System：你是一个极其严苛的数据字典定义专家。你的任务是根据上游的 DFD 和业务流程，为数据流和数据存储定义精确的字段。

你不能自由发挥，字段必须来源于业务常识和平台注入的领域模型。

Context：

- 平台约束 (EAB)：{{technicalConstraints}} // 如：主键必须是雪花算法 BIGINT
- 领域模式 (DKEE)：{{domainPatterns}} // 如：MES报工流水必须包含 ScrapQty(报废数量)

Input：

- DFD模型：{{previousDFD}} // 上一步生成的 JSON
- 流程节点：{{bpmNodes}}

Task：

请为 DFD 中的每一个【数据流】和【数据存储】生成数据字典。

必须输出严格的 JSON Schema：

{

```

"dataFlows": [
  {
    "name": "待校验报工单", // 必须与 DFD 箭头名称完全一致
    "fields": [
      {"name": "workOrderID", "type": "BIGINT", "desc": "工单ID", "isFK": true,
"refEntity": "workOrder"},
      {"name": "ReportQty", "type": "DECIMAL(18,2)", "desc": "报工数量", "isFK":
false}
    ]
  }
],
"dataStores": [
  // 同上，必须包含审计字段和租户ID
]
}

```

总结：闭环的形成

通过上述四步，我们实现了您构想的“返璞归真”：

1. **大模型的浩瀚知识被利用**：在 ScopeAgent 和 DataDictAgent 中，大模型利用其对 MES 的常识，推导出需要哪些实体和字段。
2. **大模型的自由发挥被锁死**：validator 代码像铁闸一样，只要大模型画的流程图和 DFD 对不上，或者 UI 引用了字典里没有的字段，就直接打回重生成，绝不允许带着逻辑漏洞进入下一阶段。
3. **确定性生成**：一旦走完这个受控的 8 步流水线，输出的《系统需求分析说明书》在逻辑上是绝对自治的，后续的架构设计和详细设计可以直接机械映射，彻底消灭了“一个字段返工导致一周工期修改”的灾难。